Throughput Report

Throughput Report 1
Cel i Metodologia 1
Metryki Wydajności 1
Tabela Wyników i Zbiorcze Zestawienie Metryk 1
Wizualizacja Wyników 1
Wnioski i Analiza 2
Dlaczego mmap() wygrywa przy małych blokach (M = 512 B)? 2
Dlaczego read() staje się szybszy przy dużych blokach (M >= 64 KiB)? 3
Rola pamięci podręcznej (File Cache) i narzędzia RAMMap 3
Odpowiedzi na Pytania Badawcze 3
Środowisko testowe 4

Cel i Metodologia

Celem badania jest ewaluacja wpływu mechanizmu mapowania plików do pamięci wirtualnej procesu (mmap) na czas odczytu oraz wydajność w porównaniu z klasycznym wywołaniem synchronicznym read(). Testy przeprowadzono na pliku o rozmiarze 1 GiB dla zmiennych rozmiarów bloku M. Pomiary wykonano zarówno dla warm jak i cold cache.

Metodologia czyszczenia cache na Windowsie: W celu uzyskania wiarygodnych warunków Cold Cache w systemie Windows, przed każdym pomiarem lista buforowana systemu operacyjnego (File Cache / Page Cache) byla czyszczona przy użyciu narzędzia Sysinternals RAMMap. Dzięki temu każde uruchomienie w trybie Cold Cache wymuszało fizyczny pobór danych z dysku SSD/NVMe zamiast z bufora RAM.

Metryki Wydajności

Do oceny korzyści z zastosowania mmap() wykorzystano dwie kluczowe metryki:

- **Przyspieszenie (Speedup):** Stosunek mediany czasu odczytu metodą tradycyjną do mediany czasu z użyciem mmap():

$$Speedup = \frac{T_{read}}{T_{mmap}}$$

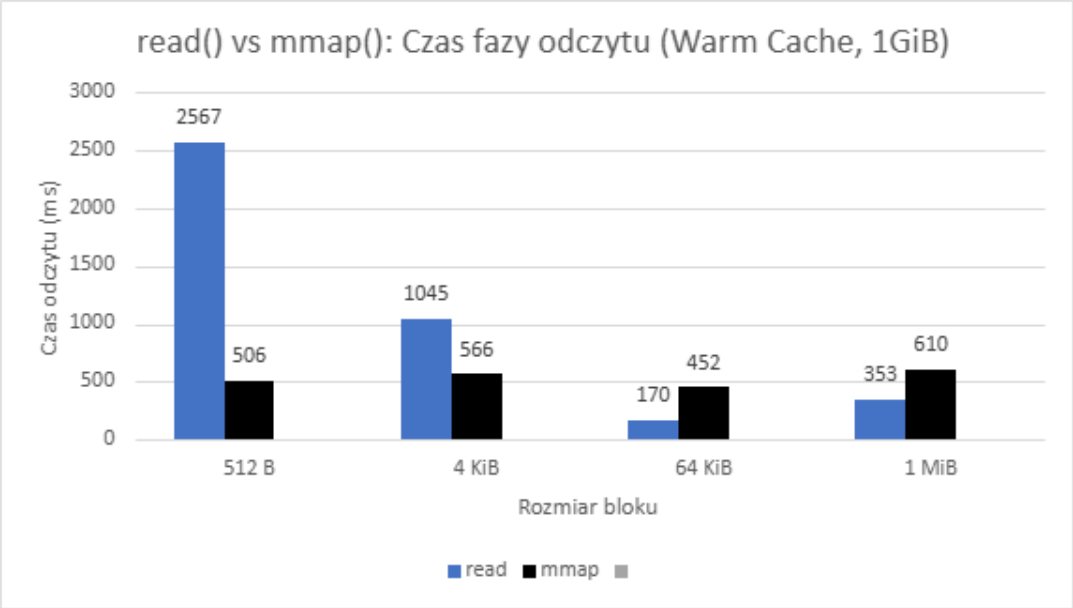
- **Procentowa redukcja czasu wykonania:** Wskaźnik określający, o ile procent skrócił się czas odczytu po wdrożeniu mmap():

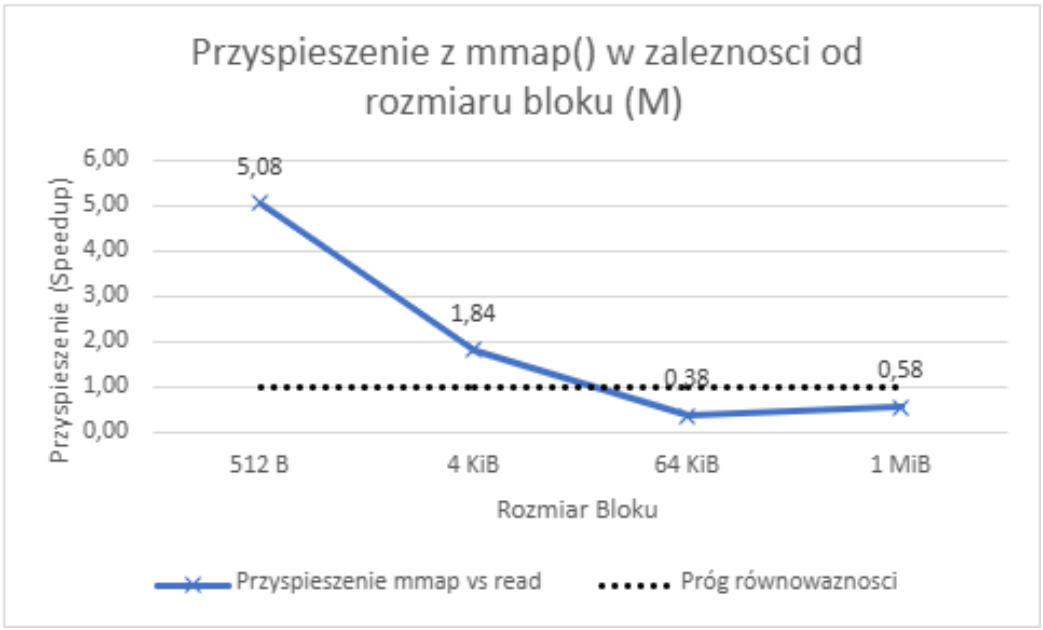
$$Reduction = \frac{T_{read} - T_{mmap}}{T_{read}} \cdot 100\%$$

Tabela Wyników i Zbiorcze Zestawienie Metryk

Rozmiar Bloku	Tryb Pamięci	read() Mediana	mmap() Mediana	Speedup	Redukcja Czasu (%)
512 B	Warm Cache	2576.2 ms	506.9 ms	5.08x	80.3%
512 B	Cold Cache	4753.0 ms	620.5 ms	7.66x	86.9%
4 KiB (4096 B)	Warm Cache	1045.7 ms	566.9 ms	1.84x	45.8%
64 KiB (65536 B)	Warm Cache	170.7 ms	452.3 ms	0.38x	-165.0%
64 KiB (65536 B)	Cold Cache	377.6 ms	624.2 ms	0.61x	-65.3%
1 MiB (1048576 B)	Warm Cache	353.7 ms	610.7 ms	0.58x	-72.7%

Wizualizacja Wyników





Wnioski i Analiza

Dlaczego mmap() wygrywa przy małych blokach (M = 512 B)?

- **Brak przełączeń kontekstu (Zero Syscalls):** Tradycyjny odczyt dla pliku 1 GiB z blokiem 512 B generuje dokładnie 2 097 152 wywołań systemowych read(). Każde przełączenie z trybu użytkownika do trybu jądra wiąże się ze zwolnieniem buforów TLB i zmianą rejestrów procesora. mmap() wykonuje mapowanie raz, przez co koszt ten spada do zera.
- **Eliminacja kopiowania pamięci (Zero-Copy):** Przy klasycznym read() jądro najpierw czyta dane z dysku/cache do własnego bufora, a następnie kopiuje je do pamięci podręcznej procesu. mmap() udostępnia bufor pamięci podręcznej bezpośrednio w przestrzeni adresowej procesu, co oszczędza gigabajty operacji kopiowania na poziomie RAM.

Dlaczego read() staje się szybszy przy dużych blokach (M >= 64 KiB)?

- **Amortyzacja narzutu wywołań systemowych:** Dla bloku M = 64 KiB liczba wywołań read() spada z 2 milionów do zaledwie 16 384. W tym momencie narzut wywołań systemowych przestaje być istotny, stanowiąc znikomy ułamek czasu pracy.
- **Narzut zarządzania pamięcią wirtualną (Page Table & Fault Overhead):** mmap() nie jest darmowy – opiera się na mechanizmie pamięci wirtualnej. Przeglądanie i aktualizacja tabel stron (Page Table Entries) oraz obsługa miękkich błędów stron (soft page faults) podczas pierwszego dostępu do stron pamięci generują stały narzut CPU. Przy dużych odczytach sekwencyjnych tradycyjny read() okazuje się bardziej optymalny pod kątem wykorzystania pamięci podręcznej L1/L2 procesora.

Rola pamięci podręcznej (File Cache) i narzędzia RAMMap

- **Stabilność mmap():** Co ciekawe, czas odczytu dla mmap() przy M = 512 B wzrasta nieznacznie między Warm Cache (506.9 ms) a Cold Cache (620.5 ms), wykazując dużą odporność na opóźnienia sprzętowe. Dla tradycyjnego read() brak cache'u (po czyszczeniu RAMMap) powoduje wydłużenie czasu z 2576.2 ms do aż 4753.0 ms (niemal dwukrotny spadek wydajności).

Odpowiedzi na Pytania Badawcze

Czas działania programu w funkcji M (przed i po poprawce)

Dla małych bloków (M = 512 B oraz M = 4 KiB) wersja mmap() osiąga spektakularne przyspieszenie względem klasycznego read(). Przy M = 512 B czas fazy odczytu spada z 2576.2 ms do 506.9 ms w Warm Cache (5.08x przyspieszenia / 80.3% redukcji czasu) oraz z 4753.0 ms do 620.5 ms w Cold Cache (7.66x przyspieszenia).

Dla dużych bloków (M >= 64 KiB) wygrywa klasyczny read() (170.7 ms vs 452.3 ms dla 64 KiB), ponieważ liczba wywołań systemowych maleje, a narzut zarządzania pamięcią wirtualną w mmap() zaczyna przeważać.

Schematy odczytu, w których mmap() zmniejsza obciążenie procesora (CPU)
mmap() redukuje narzut CPU w dwóch kluczowych sytuacjach:
(a) przy bardzo dużej liczbie małych odczytów (Heavy Small I/O, np. M = 512 B), gdzie eliminuje miliony przełączeń kontekstu user-to-kernel (syscalls) i wywołania instrukcji systemowych;
(b) przy swobodnym do pliku, gdzie unikamy ciągłych wywołań read() i ręcznej alokacji buforów, oddając zarządzenie stronowaniem do sprzętowej jednostki MMU.

Wpływ pamięci podręcznej (File Cache) na latencję
Posiadanie pliku w pamięci podręcznej (Warm Cache) skaca czas odczytu tradycyjnym read() z 4753.0 ms (Cold Cache po RAMMap) do 2576.2 ms. W przypadku mmap() latencja pozostaje bardzo stabilna niezależnie od stanu pamięci podręcznej (620.5 ms w Cold Cache vs 506.9 ms w Warm Cache), ponieważ strony są odwzorowywane bezpośrednio w przestrzeni adresowej procesu, eliminując opóźnienia związane z buforowaniem wywołań systemowych.

Środowisko testowe

Pomiary benchmarkowe zostały przeprowadzone na komputerze wyposażonym w procesor Intel Core Ultra 7 255HX, 32 GB pamięci RAM oraz 64-bitowy system operacyjny Windows. Program napisano w języku Rust i skompilowano w profilu produkcyjnym (--release). Badania wykonano przy użyciu wewnętrznego dysku SSD, uwzględniając zarówno warunki pamięci podręcznej systemu operacyjnego (Warm Cache), jak i pełne czyszczenie bufora pamięci RAM (Cold Cache) przy użyciu narzędzia RAMMap